

CARE-Judge: Calibrated, Abstaining, and Robust Evaluation via Competence-Gated Protocol Stability for LLM-as-a-Judge

Anonymous submission

Abstract

Large language models are widely used as automated judges, yet they are overconfident and sensitive to how an evaluation protocol is worded. We study whether a judge’s *protocol (in)stability* is a reliable per-instance error signal. A judge is protocol-stable on an item if its verdict survives equivalent rubric paraphrases, response-order swaps, and resampling. Across four benchmarks and five judges, we find that instability is a robust predictor of judge error: stable verdicts are up to 27 points more accurate than unstable ones. We operationalize this as **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation), which fuses the three stability families into a learned correctness calibrator and then accepts or abstains under an explicit risk budget set by an exact Clopper–Pearson threshold.

The signal is broadly effective, and we also delimit where it is not. On judges that are accurate enough to be worth deploying, fusion improves the risk–coverage tradeoff and lifts accepted accuracy well above the raw judge. The one regime in which it adds nothing is a judge with too little task competence: a very small or near-random judge lacks the knowledge to tell better from worse, so its verdicts hover around chance and their stability is uncorrelated with correctness. CARE-Judge detects this case automatically, driving coverage to zero and abstaining rather than certifying an untrustworthy judge. A negative control and an error decomposition confirm that this competence floor, not wording noise, is what limits the signal. The result is a selective evaluator that is reliable across most judges and, just as usefully, reports when a judge is too weak to be trusted at all.

1 Introduction

Large language models (LLMs) are now widely deployed as automated judges. They score RLHF outputs (Ouyang et al. 2022), rank chatbot responses (Zheng et al. 2023), and evaluate generation quality. Yet they make systematic errors, exhibit positional biases, and change their verdicts when the evaluation protocol is reworded, all while expressing high confidence (Li et al. 2024; Wang et al. 2023; Wei et al. 2024).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Prior work controls this risk explicitly but through a *single* confidence source: Jung, Brahman, and Choi (2025) (Trust-or-Escalate) use simulated-annotator agreement with cascaded selective evaluation, and Badshah, Emami, and Sajjad (2026) (SCOPE) aggregate both response orderings into a permutation-invariant conformal score. Neither exploits the broader observation that sensitivity across *multiple* semantically equivalent evaluation protocols is itself an error signal. We build on their selective-risk machinery, a held-out Clopper–Pearson threshold (Section 2). Because our threshold is chosen data-dependently, we treat the risk criterion as an empirical target and validate it directly by measured violation rates.

We investigate a complementary hypothesis: **a judgment that is unstable under semantically equivalent evaluation protocols is more likely to be wrong**. Concretely, if an LLM judge produces different rankings when (i) the rubric is paraphrased, (ii) the response order is swapped, or (iii) the judgment is re-sampled at nonzero temperature, the resulting verdict is less trustworthy. We call this property *protocol stability*. This holds broadly, but it rests on a precondition. Stability tracks correctness only when the judge is competent enough on the task to tell a better answer from a worse one. A judge that lacks this competence sits near chance and applies the same heuristic regardless of the protocol (for example a length or fluency preference) (Xu et al. 2026); its verdicts are then stable but no more accurate than a coin flip, and stability carries no information about correctness. We therefore treat protocol instability as a reliable error signal on competent judges and identify the competence floor below which it stops being useful.

We organize our study around three questions: (*RQ1*) does per-instance protocol stability predict judge error? (*RQ2*) does fusing protocol-stability signals improve the risk–coverage tradeoff over standard confidence and single-signal baselines at a matched budget? (*RQ3*) how low can a judge’s task competence fall before the signal stops being useful, and does the method detect that case on its own?

Building on this insight, **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation; Figure 1) collects the three protocol-stability signal families per item, learns a correctness calibrator $\hat{p}(\text{correct} \mid \text{features})$ on a training split, selects an acceptance threshold on a disjoint calibration split via an empirical prefix search certified with an exact Clopper–

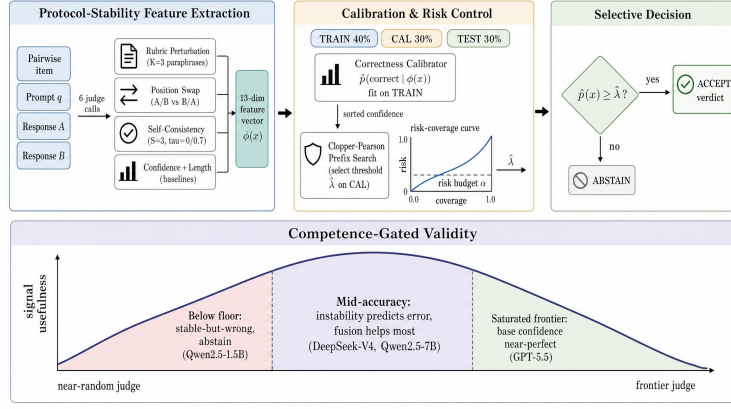


Figure 1: Overview of CARE-Judge. Six judge calls per item yield three protocol-stability feature families (rubric perturbation, position swap, self-consistency) together with confidence and length baselines ($\phi(x)$). A correctness calibrator fit on TRAIN and an exact Clopper–Pearson threshold $\hat{\lambda}$ selected on CAL produce an accept/abstain decision on held-out TEST. The lower panel summarizes when the signal is useful: it improves selective judging across competent judges, adding the most for mid-accuracy judges and little for an already-saturated frontier judge, while a judge below the competence floor sits near chance, carries no usable signal, and is abstained on automatically.

Pearson bound (Jung, Brahman, and Choi 2025; Clopper and Pearson 1934), and reports selective risk, coverage, and violation rates on a held-out test split.

We evaluate CARE-Judge on five judges that span a wide range of judge accuracy: two API judges (DeepSeek-V4 and GPT-5.5) and a same-family Qwen2.5-1.5B/7B ladder, all on four benchmarks, plus a controlled 1.5B/7B/14B size ablation on JudgeBench (Section 4.6). The four benchmarks are JudgeBench (Tan et al. 2024), RewardBench (Lambert et al. 2024), TL;DR (Stiennon et al. 2020), and LMArena (Zheng et al. 2023), giving more than 22,000 pairwise evaluations. One theme runs through the results. Protocol instability predicts judge error, and rubric-stable verdicts are up to 27 points more accurate (RQ1). Fusing the signals improves selective judging on judges that are competent enough to deploy, with the largest gains in the mid-accuracy regime (seven of eight pre-specified pairs, up to +7.6 AUROC over the best single signal) and smaller gains once a frontier judge is already saturated (RQ2). The exception is a judge below a competence floor: a very small or near-random judge sits at chance, its stability no longer tracks correctness, and CARE-Judge drives coverage to zero and abstains rather than certify it (RQ3).

Our contributions are:

1. We show that protocol instability is a broadly reliable predictor of judge error, and we characterize the *competence floor* below which it fails: a judge with too little task competence sits near chance, so its stability no longer tracks correctness (Sections 4.2, 4.4).
2. We fuse three protocol-perturbation families (rubric paraphrase, response permutation, and stochastic resampling) into a single per-instance accept/abstain rule under an explicit risk budget, and show it gains most in the mid-accuracy regime over confidence, single-signal, and ensemble baselines at a matched budget (Section 4).

An anonymized code and feature-trace archive accompanies this submission.

2 Related Work

LLM-as-a-Judge. Pairwise LLM judging is now a standard scalable evaluation paradigm (Zheng et al. 2023), and surveys document its position, length, and self-enhancement biases (Li et al. 2024); mitigations include swap-and-compare and debate (Wang et al. 2023), and prompt/rubric-template sensitivity is well established (Wei et al. 2024). Two recent findings directly motivate our design. Xu et al. (2026) give a mechanistic account showing judge biases arise from shared internal heuristics, so stability can be a warning signal but never a correctness guarantee, and verbalized confidence can be unfaithful. Li et al. (2025a) document preference leakage, which underscores the need for strictly disjoint splits when calibrating any judge-reliability estimator.

Prompt-Sensitivity, Consistency, and Per-Instance Judge Reliability. A growing body of work studies how LLM outputs shift under semantically equivalent prompt, rubric, or ordering perturbations, and how such shifts bear on per-instance reliability. Hua et al. (2025) ask whether paraphrase sensitivity is a flaw or an artifact; Guan et al. (2025) isolate the input-order effect; Anghel et al. (2025a) build a pairwise meta-evaluator diagnosing judge instability; and Huang, Sun, and Yang (2026) use prompt perturbation to stabilize pairwise evaluation. Closer to the per-instance regime, Gupta and Kumar (2026) diagnose judge reliability with conformal prediction sets, Choi et al. (2026) apply item-response theory, and Li et al. (2025b) ensemble prompt variants to raise accuracy. Ling et al. (2025) caution that abstention can itself be a prompt artifact, a caveat we address via competence-gating (Section 4.4).

These lines use protocol sensitivity differently from us. Prior work diagnoses judges globally (Hua et al. 2025; Guan

et al. 2025; Anghel et al. 2025a), builds per-instance reliability sets for analysis (Gupta and Kumar 2026; Choi et al. 2026), or ensembles prompts to improve accuracy (Li et al. 2025b; Huang, Sun, and Yang 2026). We instead fuse three perturbation families into a single calibrated accept/abstain decision under an explicit risk budget, and we characterize the competence floor none of them reports: the signal is broadly reliable but loses its footing once a judge is too weak to tell better from worse, at which point our method abstains automatically.

Selective Prediction and Abstention. Selective classification (Geifman and El-Yaniv 2017; El-Yaniv and Wiener 2010) lets a predictor abstain on uncertain inputs; recent LLM work spans abstention surveys (Wen et al. 2025), uncertainty-calibrated fine-tuning (Krishnan, Khanna, and Tickoo 2024), selective conformal uncertainty with finite-sample error control (Wang et al. 2025), and learned conformal abstention policies (Tayebati et al. 2025). We differ by using *evaluation-protocol stability* as the uncertainty source, specific to judging and requiring neither model internals nor a learned controller.

Risk-Controlled Selective Judging. Our work is closest to a recent line of risk-controlled selective LLM judging. Jung, Brahman, and Choi (2025) (Trust-or-Escalate) proposed cascaded selective evaluation with human-agreement guarantees using Simulated Annotators plus Clopper–Pearson risk control; concurrently Badshah, Emami, and Sajjad (2026) (SCOPE) aggregate both response orderings into a permutation-invariant conformal score, and Sheng et al. (2025) apply conformal prediction to judge-uncertainty intervals. CARE-Judge builds on this selective-risk apparatus (an exact Clopper–Pearson threshold on a held-out split) and contributes the confidence signal. Where Trust-or-Escalate relies on simulated-annotator agreement and SCOPE on bidirectional probability alone, we fuse rubric, position, and self-consistency instability into one calibrator and characterize the accuracy regimes where such signals help or fail, making CARE-Judge a multi-view generalization complementary to any single signal. The selection stage is orthogonal and could be coupled with conditional-guarantee frameworks such as selective conformal risk control (Xu, Guo, and Wei 2025) and online selective conformal prediction with FCR control (Bao et al. 2024); our data-dependent threshold, validated by violation rates, is a natural point of integration for such guarantees.

Judge Benchmarks and Rubric-Based Evaluation. We use JudgeBench (Tan et al. 2024) and RewardBench (Lambert et al. 2024) as hard, objectively-labeled benchmarks and TL;DR (Stiennon et al. 2020) and LMArena (Zheng et al. 2023) as subjective-preference benchmarks. Prior rubric work treats rubrics as fixed criteria to align with: Hong et al. (2026) align judges with human scoring rubrics and Anghel et al. (2025b) propose a rubric-driven multi-metric framework. A recent rubric-as-specification line makes the criteria themselves the object of design (Roy et al. 2026; Rao and Callison-Burch 2026). These aim to write better rubrics. We instead treat *rubric sensitivity*, how much a ver-

dict changes under equivalent paraphrases of a fixed criterion, as an uncertainty signal. This is a complementary use in which paraphrase-invariance, not rubric content, is the quantity of interest. Relative to the work above, our narrower gap is the specific combination of rubric-paraphrase instability as a per-instance acceptance signal within a risk-controlled selective-judging framework, which we validate empirically.

Cost-Aware Cascades. Routing easy queries to cheap models and escalating hard ones is a well-established cost-reduction strategy; Zellinger, Liu, and Thomson (2025) study LLM cascades with early abstention and formal cost analysis. Our exploratory cascade experiment (Appendix) applies the same idea to judge routing and defers a formal cascade-level guarantee to future work.

3 Method

CARE-Judge converts an LLM judge f into a *selective evaluator* $(\hat{f}, \hat{c})$ that either outputs a judgment $\hat{f}(x)$ or abstains, based on a confidence measure $\hat{c}(x)$ and a risk-controlled threshold $\hat{\lambda}$. The framework has three components: (1) multi-signal protocol-stability feature extraction, (2) learned correctness calibration, and (3) risk-controlled threshold selection with an exact Clopper–Pearson bound, validated by measured risk-violation rates.

3.1 Protocol-Stability Features

Given a pairwise evaluation item $x = (q, a_1, a_2)$ consisting of a prompt q and two candidate responses, we issue multiple judge calls under semantically equivalent evaluation protocols and measure the stability of the resulting verdicts. Let $f(x; r, \tau)$ denote the judge’s output (winner $\in \{A, B\}$ and confidence $\in [0, 1]$) given rubric r and sampling temperature τ .

Rubric Perturbation Stability. We define a set of $K = 3$ semantically equivalent rubric variants $\{r_1, \dots, r_K\}$ that are intended to express the same evaluation criteria in different wording (e.g., “Prefer the more correct response” vs. “Choose the answer a careful evaluator would prefer”); all variants are listed verbatim in the Appendix. For each item, we collect verdicts $\{v_1, \dots, v_K\}$ where $v_k = f(x; r_k, 0)$. We compute:

$$\text{rubric_vote_share} = \max_{w \in \{A, B\}} \frac{|\{k : v_k = w\}|}{K}, \quad (1)$$

$$\text{rubric_flip} = \mathbb{1}[|\{v_1, \dots, v_K\}| > 1], \quad (2)$$

where $\text{rubric_flip} = 1$ if any rubric variant produces a different winner. The intuition is that a verdict that changes under paraphrase is *protocol-dependent* and not robust to the stated specification. We avoid the stronger claim that it is “content-irrelevant,” since a paraphrase can also shift the weight a judge places on competing criteria; a negative-control experiment with intentionally non-equivalent rubrics (Section 4.2) shows that on a mid-accuracy judge, non-equivalent rubrics produce $2.2\times$ higher flip rates than equivalent ones, so equivalent-rubric flips reflect genuine instability; on a near-random judge the two conditions are indistinguishable, consistent with the signal being uninformative below the competence threshold.

Position-Swap Consistency. We judge the item in both response orderings: (q, a_1, a_2) and (q, a_2, a_1) . Let $v_{\text{orig}} = f(x; r_1, 0)$ and $v_{\text{swap}} = f(\text{swap}(x); r_1, 0)$, where swap reverses the response order. We map v_{swap} back to the original coordinate system and compute:

$$\text{swap_consistency} = \mathbb{I}[\text{map}(v_{\text{swap}}) = v_{\text{orig}}]. \quad (3)$$

This probes positional bias: a judge that reverses its verdict under response reordering is unreliable.

Self-Consistency. We form a set of S judgments under the base rubric that pools the deterministic base verdict ($\tau=0$) with $S-1$ stochastic re-samples at temperature $\tau > 0$, and compute the majority vote share:

$$\text{self_vote_share} = \max_w \frac{|\{s : f(x; r_1, \tau_s) = w\}|}{S}. \quad (4)$$

In all reported experiments $S = 3$ (one $\tau=0$ base verdict and two $\tau=0.7$ samples). An optional adaptive early-stopping rule to reduce cost was disabled in the main experiments (Appendix C).

Feature Vector. CARE-Judge uses six raw judge calls per item: one base ($\tau=0$), two self-consistency samples ($\tau=0.7$), one position swap, and two additional rubric paraphrases (the base rubric doubling as the third rubric verdict). From these it forms a 13-dimensional feature vector grouped into three families (full definitions in Appendix E, Table 5),

$$\begin{aligned} \phi(x) = [& \underbrace{\text{swap_consistency, swap_conf_gap}}_{\text{protocol: position}}, \\ & \underbrace{\text{rubric_vote_share, rubric_entropy, rubric_flip}}_{\text{protocol: rubric}}, \\ & \underbrace{\text{self_vote_share, self_entropy}}_{\text{protocol: self-consistency}}, \\ & \underbrace{\text{confidence, base_conf, mean_conf, std_conf}}_{\text{confidence}}, \\ & \underbrace{\text{length_gap, score_margin}}_{\text{length}}]. \end{aligned} \quad (5)$$

The three protocol-stability families (position, rubric, self-consistency) are the contributed signal; the confidence and length families are standard baselines the calibrator may exploit where informative. We exclude the simulated-annotator signal from ϕ (a disabled $N=0$ feature would only add a spurious degree of freedom) and instead report Trust-or-Escalate as a dedicated, leakage-controlled baseline in the head-to-head comparison (Section 4.3, Appendix C).

3.2 Correctness Calibration

We fit a calibrator $\hat{p}(x) = P(\hat{f}(x)=y \mid \phi(x))$ predicting the probability that the judge’s verdict is correct given the features (y the reference label), supporting logistic, isotonic, and gradient-boosted variants. It is fit on a *training split* D_{train} disjoint from both the threshold-selection and evaluation sets.

3.3 Empirical Risk-Targeted Threshold Calibration

Given a risk tolerance α and error level δ , we select an acceptance threshold $\hat{\lambda}$ on a *calibration split* D_{cal} (disjoint from D_{train}) targeting the calibration-conditional criterion

$$P\left(P\left(\hat{f}(x) \neq y \mid \hat{c}(x) \geq \hat{\lambda}\right) \leq \alpha\right) \geq 1 - \delta, \quad (6)$$

with the outer probability over the calibration draw. We select $\hat{\lambda}$ by an *empirical prefix search*: sort D_{cal} by descending confidence and, among prefixes of size $\geq m$, take the largest whose $(1-\delta)$ upper bound on accepted-set error is $\leq \alpha$. Because the acceptance set is chosen data-dependently this is not exact family-wise control; we treat Eq. (6) as a design target and validate it empirically via the realized violation frequency $\text{Pr}_{\text{seed}}(\hat{R}_{\text{test}} > \alpha)$ over many held-out splits (Section 4). We use the exact one-sided Clopper–Pearson upper bound (Clopper and Pearson 1934):

$$\text{CP}_{\text{upper}}(e, n, \delta) = \sup\{p : P(\text{Bin}(n, p) \leq e) \geq \delta\}, \quad (7)$$

where e is the number of errors among the n accepted calibration items, computed via bisection on the binomial CDF. The complete train/calibrate/test procedure, including the minimum-acceptance floor m that prevents certifying on too few points, is given as Algorithm 1 in Appendix A. The selective evaluator is then applied to a *held-out test split* D_{test} :

$$(\hat{f}, \hat{c})(x) = \begin{cases} \hat{f}(x) & \text{if } \hat{c}(x) \geq \hat{\lambda}, \\ \emptyset & \text{otherwise (abstain).} \end{cases} \quad (8)$$

All reported metrics (coverage, selective risk, accepted accuracy, calibration error) are measured on the held-out D_{test} ; the empirical violation rate over many splits is our primary evidence that Eq. (6) holds in practice. As an application, the same threshold rule extends to a cost-aware cheap-to-strong *cascade* with per-tier conditional calibration and a $\delta_t = \delta/T$ union-bound error budget; we treat it as exploratory and defer it to Appendix F.

4 Experiments

4.1 Experimental Setup

Datasets. We use four benchmarks spanning objective correctness and subjective preference: JudgeBench (Tan et al. 2024) (620 objectively-labeled pairs over knowledge/reasoning/math/coding), RewardBench (Lambert et al. 2024) (chosen/rejected pairs over chat/safety/reasoning), TL;DR (Stiennon et al. 2020) (summarization preference), and LMArena (Zheng et al. 2023) (Chatbot-Arena human preference). We scale each subjective benchmark to 2,000 pairs (dropping ties) so the 40/30/30 split gives calibration/test folds of several hundred items, enough for the exact Clopper–Pearson bound to certify nonzero coverage. We redistribute only derived feature traces, not raw text, under each benchmark’s license.

Judge Models. We use five judges. Throughout, “below-threshold”, “mid-accuracy”, and “frontier” refer to a judge’s *raw agreement on the benchmark at hand* (near chance, 0.65–0.80, and above, respectively), not to the underlying model’s

general capability; the same model can fall in different bands on different benchmarks (Section 4.6). Two API judges cover the mid-to-frontier range as judges: **DeepSeek-V4** in non-thinking `deepseek-chat` mode (its judge agreement lands in the mid band; the reasoning `v4-pro` mode is too token-intensive for six-call collection) and **GPT-5.5** (frontier agreement, default reasoning effort). The other three form a controlled same-family ladder, **Qwen2.5-1.5B/7B/14B-Instruct**, holding recipe, tokenizer, and prompt fixed so that only parameter count varies. This isolates a size-driven trend in judge agreement from the broader (confounded) API-vs-open comparison. Base confidence is verbalized confidence, since log-probabilities are not universally exposed. Exact snapshot identifiers, dates, decoding settings, and invalid-output handling are in Appendix C. (GLM-5.2 was attempted but behaves as a slow reasoning model on the available endpoint, so we omit it.)

Feature Collection, Protocol, and Baselines. Each item uses six judge calls: one base ($\tau=0$), two self-consistency samples ($\tau=0.7$, pooled to $S=3$), one position swap, and two rubric paraphrases ($K=3$ with the base rubric). Unparseable model outputs are retried once, then recorded as an invalid state that counts as a disagreement (lowering, never inflating, stability); genuine parse-invalid rates are $< 3\%$ everywhere. Separately, a subset of GPT-5.5 API calls failed with transient service errors (rate-limit/quota) rather than model outputs; these are not judge behavior, so the affected items are excluded from GPT-5.5’s analysis and we report its per-benchmark retained N in Table 1 (Appendix C). Ties are dropped on the preference benchmarks. All results use a strict 40/30/30 train/calibration/test split (fit calibrator / select threshold / report), 20 seeds, $\alpha=0.15$, $\delta=0.10$, exact Clopper–Pearson. At the matched six-call budget we compare Raw, CARE (logistic fusion), the single signals base/rubric/swap (a budget-matched reimplementation of SCOPE’s bidirectional signal (Badshah, Emami, and Sajjad 2026))/self, oracle best-single, and a budget-matched Trust-or-Escalate simulated-annotator baseline (Jung, Brahman, and Choi 2025). Budget parity is enforced by scoring every method from the same six per-item calls: a single signal is a projection of that shared call set onto one view (the SCOPE-style swap score reads the base and swapped verdicts; self-consistency reads the two $\tau=0.7$ samples), and a signal needing fewer calls is not charged for the unused ones, so any AUROC gap reflects information content, not spend.

4.2 Main Results

Table 1 presents the main held-out results across all model-dataset combinations.

Protocol instability and judge error (RQ1). Across all four primary judges, rubric- and position-stable verdicts are more accurate than unstable ones on almost every benchmark (Fig. 2, Appendix I). For DeepSeek-V4 the rubric gap ranges from $+4.1\text{pp}$ (JudgeBench) to $+27.4\text{pp}$ (RewardBench). It persists for the frontier GPT-5.5 ($+27.0\text{pp}$ on RewardBench) but its unstable subset is small (rubric-flip $< 8\%$), bounding how much the signal can add once near-saturated (RQ2).

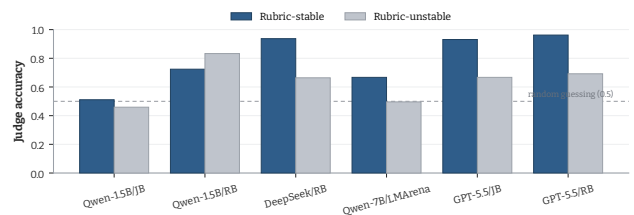


Figure 2: Judge accuracy on rubric-stable (blue) vs. rubric-unstable (grey) subsets, ordered by judge accuracy; the dashed line marks chance (0.5). For the mid-accuracy and frontier judges the stable subset is clearly more accurate, so stability is informative. For the below-competence Qwen2.5-1.5B, both subsets sit near chance and stability no longer separates correct from incorrect verdicts (analyzed in §4.2, §4.4).

The only judge where the gap does not hold is the below-competence Qwen2.5-1.5B on RewardBench, where both subsets are near chance and stable verdicts are not more accurate ($-10.9/-11.0\text{pp}$): with too little competence to tell better from worse, this judge cannot produce an informative signal, the case we examine next.

Negative control. To rule out that equivalent-rubric flips are wording noise, we contrast the $K=3$ equivalent paraphrases against an *intentionally non-equivalent* rubric set (brevity/thoroughness/formality) on JudgeBench (two judges; full counts in Appendix C). On DeepSeek-V4 the non-equivalent set flips $2.2\times$ more often (28.0% vs. 12.8%), so equivalent-rubric flips are a conservative lower bound on instability, not artifacts; the near-random Qwen-1.5B shows no such ordering, being insensitive to what the rubric asks. The contrast thus separates genuine instability from paraphrase noise, and the same accuracy dependence reappears here.

Where fusion helps (RQ2). RQ2 asks not whether fusion helps on average, which RQ1 predicts it should not, but whether it helps in the mid-accuracy band where the signal is informative. Two recipe-independent judges occupy that band: DeepSeek-V4 (a strong general model whose judge agreement lands mid) and the open Qwen2.5-7B. On DeepSeek-V4 fusion beats the best single signal by up to $+6.3$ AUROC (LMArena, $0.615 \rightarrow 0.678$), with paired-bootstrap 95% CIs over 20 seeds excluding zero on three of four benchmarks (JudgeBench, $+1.0$, includes zero). On Qwen2.5-7B it is positive on three benchmarks (up to $+7.6$ on RewardBench) but falls 3.0 below the swap signal on JudgeBench, where a 7B open judge is near chance. Seven of the eight mid-accuracy pairs thus show positive gains, as competence-gating predicts.

On the frontier GPT-5.5, fusion ties its already-strong base confidence to within noise (≤ 0.02 AUROC on JudgeBench and RewardBench), consistent with little instability left to exploit; below the threshold no signal exceeds ≈ 0.58 on Qwen-1.5B’s subjective benchmarks.

Regimes are assigned by raw agreement alone before any fusion result is seen, so the pre-specified test over the eight

Table 1: Held-out selective evaluation and matched-budget signal comparison (logistic fusion, 20 seeds, $\alpha=0.15$, $\delta=0.10$, exact Clopper–Pearson, strict 40/30/30 split). Judges are grouped by family—the open Qwen2.5 ladder (1.5B, 7B) followed by the API judges (DeepSeek-V4, GPT-5.5); by raw judge agreement these span below-threshold (Qwen2.5-1.5B), mid-accuracy (Qwen2.5-7B, DeepSeek-V4), and frontier (GPT-5.5). N is items (test fold $0.3N$); coverage/accepted-accuracy are pooled across seeds; “Viol.” is the seed fraction with held-out accepted error above α ; coverage 0.000 (Acc. “—”) means no confidence level meets the risk constraint, so CARE abstains. The right block reports matched six-call-budget AUROC per single signal versus the **fused** calibrator (best per row **bold**); “swap” is the SCOPE-style bidirectional signal, “best-1” the oracle best single signal on train.

Model	Dataset	Selective ($\alpha=0.15$)				Matched-budget AUROC				
		N	Cov.	Acc.	Viol.	base	swap	rubric	best-1	fusion
Qwen2.5-1.5B	JudgeBench	620	0.000	—	0.00	0.497	0.534	0.534	0.534	0.523
	TL;DR	2000	0.000	—	0.00	0.503	0.570	0.535	0.570	0.583
	RewardBench	2000	0.516	0.880	0.10	0.609	0.425	0.436	0.609	0.713
	LMarena	2000	0.000	—	0.00	0.515	0.563	0.553	0.563	0.578
Qwen2.5-7B	JudgeBench	619	0.000	—	0.00	0.510	0.578	0.511	0.578	0.548
	TL;DR	2000	0.000	—	0.00	0.549	0.616	0.515	0.616	0.636
	RewardBench	2000	0.987	0.877	0.00	0.611	0.605	0.566	0.611	0.687
	LMarena	1952	0.000	—	0.00	0.522	0.588	0.539	0.588	0.599
DeepSeek-V4	JudgeBench	620	0.014	0.923	0.00	0.593	0.561	0.522	0.593	0.603
	TL;DR	2000	0.085	0.829	0.40	0.537	0.616	0.549	0.616	0.675
	RewardBench	2000	0.999	0.926	0.00	0.787	0.606	0.591	0.787	0.831
	LMarena	2000	0.077	0.881	0.05	0.604	0.615	0.562	0.615	0.678
GPT-5.5	JudgeBench	620	0.994	0.915	0.00	0.845	0.695	0.576	0.845	0.834
	TL;DR	2000	0.218	0.882	0.20	0.650	0.629	0.537	0.650	0.667
	RewardBench	1999	0.997	0.961	0.00	0.904	0.757	0.546	0.904	0.888
	LMarena	1999	0.126	0.871	0.30	0.634	0.571	0.548	0.634	0.645

mid-accuracy pairs (seven positive) is not fit post hoc. Pooling all 17 pairs, including regimes where fusion cannot help by construction, dilutes this to a one-sided Wilcoxon $p=0.087$ (11/17 wins). In selective terms, at $\alpha=0.15$ CARE raises accepted accuracy above raw by abstaining on the risky tail (e.g. DeepSeek-V4/TL;DR: 8.5% coverage at 82.9% accuracy, raw 67.9%), while a saturated judge is accepted almost entirely (GPT-5.5/RewardBench 99.7% at 96.1%).

The competence floor, and abstention as the correct response (RQ3). When a judge has too little competence on a task, protocol stability stops carrying information. Where Qwen2.5-1.5B is near chance (JudgeBench 0.49, TL;DR/LMArena 0.58), its stable verdicts are no more correct than its unstable ones, because a judge that cannot tell better from worse produces the same answer under every protocol for the wrong reason. The right behavior here is not to trust the judge but to step back, and this is exactly what CARE-Judge does: it accepts zero examples and abstains. Competence is per task rather than per model: on RewardBench the same 1.5B judge is more accurate (raw 0.77), and there fusion behaves as in the mid-accuracy regime (0.713 AUROC, 51.9% coverage at 88.0%). The two behaviors are complementary. On a judge worth deploying, CARE-Judge raises accepted accuracy; on a judge too weak to be trusted, it withholds certification instead of manufacturing false confidence. We quantify the near-chance floor in Section 4.4.

Is the gate a deployed component or a post-hoc label? We separate two senses of “competence-gating.” The three-

band naming (below-threshold / mid-accuracy / frontier) is a post-hoc categorization for organizing the analysis, not an online detector. The *operational* gate, accept versus abstain, is however applied automatically: on exactly the pairs we would call below-threshold, no calibration-split confidence level meets the risk bound, so the threshold is $+\infty$ and coverage collapses to 0.000 with no regime label supplied (Qwen2.5-1.5B on JB/TL;DR/LMArena; Qwen2.5-7B on the same three). The system abstains where the signal is uninformative by consequence of the risk criterion, not of our labeling. CARE still needs task-specific labels to fit and calibrate, and these do not transfer freely (Appendix G), so it is a per-task reliability layer. Versus a supervised reward model on the same labels, its value is abstention under a measured risk target with interpretable signals, plus the automatic zero-coverage collapse that flags an untrustworthy judge, which a plain reward model does not surface.

4.3 Head-to-Head System Comparison

We compare CARE against the two prior selective-judging systems, SCOPE and Trust-or-Escalate, at the same six-call budget (full 17-pair table in Appendix D). CARE attains the highest AUROC on 14 of the 17 pairs, including all eight DeepSeek/GPT pairs; the three exceptions are Qwen judges on JudgeBench, where SCOPE’s bidirectional score is marginally higher near the accuracy floor. The margin over SCOPE is large where position flips are rare (e.g. DeepSeek-V4 0.831 vs. 0.606 on RewardBench): a frontier judge rarely flips under reordering, so its consistency sig-

nal is weak, whereas fusion also leverages base confidence. Trust-or-Escalate hovers near chance (0.50–0.53) at $N=0$ dedicated annotators. No single signal is uniformly best; the learned fusion delivers robustness across the spectrum.

4.4 The Competence Floor

To see why the signal loses its footing on a weak judge, we decompose Qwen-1.5B’s 620 JudgeBench verdicts: 65% of its 314 errors are *rubric-stable-wrong* (203/314), and only 51.1% of its 415 rubric-stable verdicts are correct, essentially chance. A judge this weak does not disagree with itself under perturbation, but its agreement is empty: it applies one heuristic to every item, consistent with mechanistic analyses of judge bias (Xu et al. 2026; Panickssery, Bowman, and Feng 2024), so stability and correctness simply decouple.

What the stable-but-wrong errors look like. These 203 items reflect a confidently held wrong heuristic, not noise: their mean verbalized confidence (0.797) is indistinguishable from that on correct verdicts (0.785), so self-reported confidence is useless exactly where a stability signal is meant to help, and it still fails. Such a gap between *stated* and *revealed* preference is well documented in choice modeling (Yu and Jayakrishnan 2018), motivating our use of behavior under perturbation rather than self-reported confidence. Nor are they a length artifact (55.2% longer-response picks vs. 51.6% overall; Appendix K), so the heuristic is semantic. This is the regime in which CARE-Judge should, and does, abstain: with no informative signal available, driving coverage to zero is the safe outcome rather than a failure of the method.

4.5 Ensemble Baselines and Feature-Group Ablation

Against two ensemble baselines (majority/weighted vote) and a by-group feature ablation (full tables in Appendix J), CARE is best or tied-best in 12 of 16 pairs. Weighted vote is competitive for GPT-5.5 (it uses confidence implicitly) but has no abstention mechanism: a vote must rule on every item, whereas on DeepSeek-V4/TL;DR CARE abstains to 8.5% coverage at 82.9% accuracy versus the raw 67.9%. The group ablation confirms complementary roles: confidence dominates for GPT-5.5, protocol features carry the ladder judges, and length is near-chance throughout.

4.6 Judge-Capability Dependence: A Controlled Ladder

Is the signal useful because judges are *more capable* or merely *different systems*? A controlled same-family ladder (Qwen2.5-1.5B/7B/14B) on JudgeBench varies only parameter count (Table 2): raw accuracy rises with scale (0.49 $\rightarrow$ 0.57 $\rightarrow$ 0.60) while fused AUROC rises from 1.5B to 7B (.52 $\rightarrow$.55) then plateaus at 14B (.54, within a seed-level standard deviation of 7B), so the signal saturates by the 7B scale rather than growing linearly with size. By holding recipe, tokenizer, and prompt fixed, the ladder isolates parameter count as the only varying factor; the size effect that the confounded API-vs-open comparison cannot

Table 2: Controlled size ladder on JudgeBench: same-family Qwen2.5 judges varying only in parameter count (20 seeds, 620 items). Columns report raw judge accuracy and fused AUROC (mean $\pm$ std) per size.

Judge	Params	Mean raw acc.	Fused AUROC
Qwen2.5-1.5B	1.5B	0.49	.52 $\pm$.06
Qwen2.5-7B	7B	0.57	.55 $\pm$.04
Qwen2.5-14B	14B	0.60	.54 $\pm$.04

cleanly attribute. We run it on JudgeBench’s 620 objective-labeled items (the 14B judge runs at $\approx$ 200 items/hour); a full size $\times$ recipe grid is a natural extension.

The 7B rung doubles as the second mid-accuracy judge underpinning RQ2: sharing DeepSeek-V4’s accuracy band but none of its recipe, it makes the fusion claim a recipe-independent replication, positive on three of its four benchmarks.

4.7 Robustness and Calibration

Over 20 splits on DeepSeek/TL;DR fused AUROC is 0.675 ± 0.016 (CV 2.4%); GPT-5.5 is well calibrated (ECE $\leq$ 0.06), DeepSeek-V4 looser (ECE 0.16–0.29). Sweeping $\delta \in \{0.05, 0.10, 0.20\}$ and $n_{\text{cal}} \in [200, 1000]$ leaves AUROC unchanged (0.676 ± 0.001), so CARE needs no large calibration set. Varying α from 0.05 to 0.30 traces the expected coverage/accuracy tradeoff (Table 3; risk–coverage curves in Appendix H).

5 Conclusion

We studied *protocol stability* as a per-instance reliability signal for LLM-as-a-judge. A verdict that changes under semantically equivalent rubrics, orderings, or resampling is less reliable, and this holds broadly as long as the judge is competent enough on the task for its instabilities to track difficulty. CARE-Judge turns the signal into practice by fusing the three instability families into a learned calibrator with inherited selective-risk control (Jung, Brahman, and Choi 2025; Badshah, Emami, and Sajjad 2026). It has two complementary strengths: on judges worth deploying it improves selective accuracy, with the largest gains in the mid-accuracy band and smaller gains on an already-saturated frontier judge; on a judge with too little task competence, whose verdicts sit near chance, it certifies nothing and abstains, so a weak judge is flagged rather than trusted.

Practical recommendation. Deploy protocol-stability abstention for mid-accuracy judges ($\approx$ 0.65–0.8 raw agreement), where fusion buys the largest risk-controlled coverage; the perturbation calls are optional for a frontier judge, and a judge that scores near chance on a task should be abstained on rather than used.

Limitations and Future Work. Risk control is empirical: the threshold is data-dependent and validated by measured violation rates, not a family-wise theorem, so a selective-conformal rule (Xu, Guo, and Wei 2025; Bao et al. 2024) would strengthen it. CARE needs task-specific labels and

does not transfer freely (Appendix G), and stability never implies correctness. A crossed size×recipe study, a guaranteed cascade, and pointwise scoring remain future work.

References

- Anghel, C.; Anghel, A. A.; Pecheanu, E.; Cocu, A.; Istrate, A.; and Andrei, C. A. 2025a. Diagnosing Bias and Instability in LLM Evaluation: A Scalable Pairwise Meta-Evaluator. *Information*, 16(8): 652.
- Anghel, C.; Anghel, A. A.; Pecheanu, E.; Crăciun, M.; Cocu, A.; and Niculita, C. 2025b. PEARL: A Rubric-Driven Multi-Metric Framework for LLM Evaluation. *Information*, 16(11).
- Badshah, S.; Emami, A.; and Sajjad, H. 2026. SCOPE: Selective Conformal Optimized Pairwise LLM Judging. arXiv:2602.13110. arXiv:2602.13110.
- Bao, Y.; Huo, Y.; Ren, H.; and Zou, C. 2024. CAP: A General Algorithm for Online Selective Conformal Prediction with FCR Control.
- Choi, J.; Park, S.; Cho, C.; Park, H.; and Kim, B. 2026. Diagnosing the Reliability of LLM-as-a-Judge via Item Response Theory. arXiv:2602.00521. arXiv:2602.00521.
- Clopper, C. J.; and Pearson, E. S. 1934. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika*, 26(4): 404–413.
- El-Yaniv, R.; and Wiener, Y. 2010. On the Foundations of Noise-free Selective Classification. *JMLR*.
- Geifman, Y.; and El-Yaniv, R. 2017. Selective Classification for Deep Neural Networks. In *NeurIPS*.
- Guan, B.; Roosta, T.; Passban, P.; and Rezagholizadeh, M. 2025. The Order Effect: Investigating Prompt Sensitivity to Input Order in LLMs. In arXiv:2502.04134.
- Gupta, M.; and Kumar, D. 2026. Diagnosing LLM Judge Reliability: Conformal Prediction Sets and Transitivity Violations. arXiv:2604.15302. arXiv:2604.15302.
- Hong, Y.; Yao, H.; Shen, B.; Xu, W.; Wei, H.; and Dong, Y. 2026. From Rubrics to Reliable Scores: Evidence-Grounded Text Evaluation with LLM Judges. In arXiv:2601.08654.
- Hua, A.; Tang, K.; Gu, C.; Gu, J.-Y.; Wong, E.; and Qin, Y. 2025. Flaw or Artifact? Rethinking Prompt Sensitivity in Evaluating LLMs. *Proceedings of EMNLP*, 19900–19910.
- Huang, D.; Sun, J.; and Yang, P. 2026. Prompt Perturbation for Reliable LLM Evaluation over Comparison Graphs. arXiv:2606.17634. arXiv:2606.17634.
- Jung, J.; Brahman, F.; and Choi, Y. 2025. Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement. In *ICLR*.
- Krishnan, R.; Khanna, P.; and Tickoo, O. 2024. Enhancing Trust in Large Language Models via Uncertainty-Calibrated Fine-Tuning. In arXiv:2412.02904.
- Lambert, N.; Pyatkin, V.; Morrison, J.; et al. 2024. Reward-Bench: Evaluating Reward Models for Language Modeling. In arXiv:2403.13787.
- Li, D.; Sun, R.; Huang, Y.; Zhong, M.; Jiang, B.; Han, J.; Zhang, X.; Wang, W.; and Liu, H. 2025a. Preference Leakage: A Contamination Problem in LLM-as-a-judge. arXiv:2502.01534. arXiv:2502.01534.
- Li, H.; Dong, Q.; Chen, J.; Su, H.; Zhou, Y.; Ai, Q.; Ye, Z.; and Liu, Y. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv preprint arXiv:2412.05579*.
- Li, J.; Zhang, H.; Lin, P.; Xiong, J.; and Xu, W. 2025b. Auto-Prompt Ensemble for LLM Judge. arXiv:2510.06538. arXiv:2510.06538.
- Ling, Z.; Liu, S.; Tang, Y.; Yang, J.; Fu, S.; Huang, C.; Huang, K.; Wan, Y.; Hou, Z.; and Hu, X. 2025. LLM Abstention Can Be a Prompt Artifact, in Addition to Genuine Uncertainty. arXiv:2507.16199. arXiv:2507.16199.
- Ouyang, L.; Wu, J.; Jiang, X.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *NeurIPS*.
- Panickssery, A.; Bowman, S. R.; and Feng, S. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Rao, D.; and Callison-Burch, C. 2026. Autorubric: Unifying Rubric-based LLM Evaluation. arXiv:2603.00077.
- Roy, S.; Pujari, R.; Kumarage, T.; Peris, C.; Gupta, R.; Rumshisky, A.; Natarajan, P.; and Saligrama, V. 2026. PReMISE: Policy Rubrics as Measurement Specifications for LLM Judges. arXiv:2605.30803.
- Sheng, H.; Liu, X.; He, H.; Zhao, J.; and Kang, J. 2025. Analyzing Uncertainty of LLM-as-a-Judge: Interval Evaluations with Conformal Prediction. arXiv:2509.18658. arXiv:2509.18658.
- Stiennon, N.; Ouyang, L.; Wu, J.; Ziegler, D. M.; Lowe, R.; Voss, C.; Radford, A.; Amodei, D.; and Christiano, P. F. 2020. Learning to summarize from human feedback. In *NeurIPS*.
- Tan, S.; Zhuang, S.; Montgomery, K. L.; Tang, W.; Cuadron, A.; and Wang, C. 2024. JudgeBench: A Benchmark for Evaluating LLM-Based Judges. In arXiv:2410.12784.
- Tayebati, S.; Kumar, D.; Darabi, N.; Jayasuriya, D.; Krishnan, R.; and Trivedi, A. R. 2025. Learning Conformal Abstention Policies for Adaptive Risk Management in Large Language and Vision-Language Models. arXiv:2502.06884. arXiv:2502.06884.
- Wang, P.; Li, L.; Chen, L.; Zhu, D.; Lin, B.; Cao, Y.; Liu, Q.; Liu, T.; and Sui, Z. 2023. Large Language Models are not Fair Evaluators. In arXiv:2305.17926.
- Wang, Z.; Wang, Q.; Zhang, Y.; Chen, T.; Zhu, X.; Shi, X.; and Xu, K. 2025. SConU: Selective Conformal Uncertainty in Large Language Models. 19052–19075.
- Wei, H.; He, S.; Xia, T.; Liu, F.; Wong, A.; Lin, J.; and Han, M. 2024. Systematic Evaluation of LLM-as-a-Judge in LLM Alignment Tasks: Explainable Metrics and Diverse Prompt Templates. In arXiv:2408.13006.
- Wen, B.; Yao, J.; Feng, S.; Xu, C.; Tsvetkov, Y.; Howe, B.; and Wang, L. L. 2025. Know Your Limits: A Survey of Abstention in Large Language Models. *TACL*, 13: 529–556.
- Xu, Y.; Guo, W.; and Wei, Z. 2025. Selective Conformal Risk Control. arXiv:2512.12844.

Xu, Z.; Li, S.; Liu, H.; Wang, X.; Li, S.; Song, Z.; and Chen, X. 2026. Inside the Unfair Judge: A Mechanistic Interpretability Account of LLM-as-Judge Bias. arXiv:2607.11871. arXiv:2607.11871.

Yu, J. G.; and Jayakrishnan, R. 2018. A quantum cognition model for bridging stated and revealed preference. *Transportation Research Part B: Methodological*, 118: 263–280.

Zellinger, M. J.; Liu, R.; and Thomson, M. 2025. Cost-Saving LLM Cascades with Early Abstention. arXiv:2502.09054. arXiv:2502.09054.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*.

Supplementary Material

Reviewers are not obligated to read this material (AAAI-27 policy).

A CARE-Judge Algorithm

Algorithm 1 CARE-Judge selective evaluation

```
1: Input: labeled items  $D$ ; risk  $\alpha$ , level  $\delta$ ; min-accept  $m$ 
2: Split  $D$  into disjoint  $D_{\text{train}}, D_{\text{cal}}, D_{\text{test}}$  (40/30/30)
3: Fit calibrator  $\hat{p}$  on  $\{(\phi(x), \mathbb{I}[\hat{f}(x)=y])\}_{x \in D_{\text{train}}}$ 
4: Sort  $D_{\text{cal}}$  by  $\hat{p}$  descending:  $c_{(1)} \geq \dots \geq c_{(n)}$ 
5:  $\hat{\lambda} \leftarrow \infty$  {accept nothing by default}
6: for  $i = m$  to  $n$  do
7:    $e_i \leftarrow \#\{j \leq i : \hat{f}(x_{(j)}) \neq y_{(j)}\}$  {errors in top- $i$  prefix}
8:   if  $\text{CP}_{\text{upper}}(e_i, i, \delta) \leq \alpha$  then
9:      $\hat{\lambda} \leftarrow \hat{p}(x_{(i)})$  {largest certified prefix so far}
10:  end if
11: end for
12: Output: on  $D_{\text{test}}$ , accept  $x$  iff  $\hat{p}(x) \geq \hat{\lambda}$ , else abstain
13: Report: empirical violation rate  $\Pr_{\text{seed}}(\hat{R}_{\text{test}} > \alpha)$  over splits
```

B Alpha-Sweep Risk–Coverage Tradeoff

Table 3 traces how coverage and accepted accuracy vary with the risk budget α for the two API judges, extending the single $\alpha=0.15$ operating point of Table 1 to the full 0.05–0.30 range.

C Reproducibility Details

Judge model configurations. GPT-5.5 was queried through an OpenAI-compatible endpoint at its default reasoning effort; DeepSeek-V4 was queried in non-thinking (deepseek-chat) mode because the reasoning v4-pro mode is too token-intensive for six-call-per-item feature collection. The controlled capability ladder uses Qwen/Qwen2.5-1.5B-Instruct, Qwen/Qwen2.5-7B-Instruct, and Qwen/Qwen2.5-14B-Instruct run locally in fp16 on a single 24 GB GPU, with identical prompts, chat template, and decoding settings so that only parameter count varies. All deterministic verdicts use temperature 0; self-consistency samples use temperature 0.7 with `max_new_tokens=80`. Base confidence is the model’s verbalized confidence, since token log-probabilities are not exposed by all endpoints. We pin and release the exact API model snapshot identifiers and dates, the local model commit hashes, all prompt templates, the six-call collection code, per-seed split indices, and the fitted calibrators; the AAAI reproducibility checklist is completed in the supplementary archive.

Compute budget. All local judges (the Qwen2.5 ladder) were run on a single 32 GB consumer GPU (RTX 5090). The 1.5B judge collects features at $\approx 1,500$ items/hour, the 7B at ≈ 450 /hour, and the 14B at ≈ 200 /hour; total local feature collection for the ladder was under 40 GPU-hours. Each item consumes exactly six judge calls, so feature collection for the API judges required $6 \times 6,620 \approx 40,000$ calls per API model (DeepSeek-V4 and GPT-5.5), plus 2×898 additional calls for the DeepSeek negative-control. The calibrator fit and all 20-seed analyses run in minutes on a CPU. This modest budget is intentional: CARE-Judge is designed to be reproducible on a single commodity GPU plus metered API access.

Tie and invalid handling. An output that cannot be parsed as A/B is retried once. If it is still unparseable we record it as an explicit `invalid` state rather than silently mapping it to the base verdict: an `invalid` perturbation call counts as a *disagreement* with the base verdict, so it *lowers* the corresponding stability signal and can only push an item toward abstention. We avoid mapping `invalid`→base verdict, because that would inflate measured stability (an unparseable output would be counted as agreement). Genuine parse-invalid rates (the model returned an output we could not map to A/B) are below 3% for every judge–benchmark pair. We distinguish these from *transient API-service failures* (rate-limit/quota responses that return no model output): during GPT-5.5 collection a subset of calls failed this way, and because such failures reflect infrastructure rather than judge behavior we *exclude* the affected items from GPT-5.5’s analysis rather than count them as verdicts. The retained per-benchmark sample sizes for GPT-5.5 are JudgeBench 620, TL;DR 2,000, RewardBench 1,999, and LMArena 1,999 (Table 1); all other judges retain their full sets. The exclusions total two items across all four benchmarks (1 each on RewardBench and LMArena, 0.03% of GPT-5.5’s evaluations), so they cannot materially shift any reported statistic or introduce a selection bias: the transient rate-limit failures are triggered by request timing, not by item content, and the retained accuracy on each benchmark (Table 1) is within sampling noise of the full-set raw accuracy. We therefore treat the exclusion as missing-completely-at-random.

Table 3: Alpha-sweep risk-coverage tradeoff (logistic fusion, 20 seeds *pooled* across seeds, recomputed from the per-item feature traces at each of the six α levels, consistent with the $\alpha=0.15$ column of Table 1). We show the two API judges (DeepSeek-V4, GPT-5.5), which have non-trivial coverage across this range; the near-chance ladder judges abstain at zero coverage across most of it (Table 1) and are omitted rather than shown as all-“–” rows. Each cell reports coverage / accepted accuracy; “–” denotes safe abstention (zero coverage) and “<.01” a positive coverage below 0.005. Coverage rises with the risk budget α but is not strictly monotone: because the accepted prefix is chosen data-dependently on each split, relaxing α occasionally selects a different threshold that shifts pooled coverage.

Model	Dataset	$\alpha=0.05$	$\alpha=0.10$	$\alpha=0.15$	$\alpha=0.20$	$\alpha=0.25$	$\alpha=0.30$
DeepSeek-V4	JudgeBench	–/–	.01/.96	.01/.92	.04/.88	.15/.76	.51/.72
	TL;DR	–/–	<.01/.82	.08/.83	.32/.81	.59/.77	.82/.72
	RewardBench	.78/.96	1.00/.93	1.00/.93	1.00/.93	1.00/.93	1.00/.93
	LMarena	–/–	.01/.89	.08/.87	.45/.82	.77/.77	.96/.73
GPT-5.5	JudgeBench	.52/.97	.95/.93	.99/.92	.99/.92	.99/.92	.99/.92
	TL;DR	.03/.92	.08/.89	.22/.86	.50/.82	.82/.77	.99/.73
	RewardBench	.99/.96	1.00/.96	1.00/.96	1.00/.96	1.00/.96	1.00/.96
	LMarena	<.01/.90	.02/.91	.13/.85	.34/.81	.59/.77	.88/.73

For the preference benchmarks (TL;DR, LMarena) genuine ties in the human label are dropped before evaluation; JudgeBench and RewardBench carry objective correct/incorrect labels and contain no ties.

Rubric variants and the negative control (full detail). The $K=3$ rubric paraphrases used for the rubric-stability signal are reproduced verbatim in the released prompt file. They were written to hold the evaluation target (which response is better) fixed while varying wording. The negative-control experiment summarized in Section 4.2 used an *intentionally non-equivalent* rubric set (one emphasizing brevity, one thoroughness, one formality) on both a mid-accuracy judge (DeepSeek-V4) and a near-random judge (Qwen2.5-1.5B), both on JudgeBench. The full per-condition counts are as follows. **DeepSeek-V4**: equivalent set flip rate 12.8% (46/359) vs. non-equivalent 28.0% (151/539), a $2.2\times$ ratio. **Qwen2.5-1.5B**: equivalent 31.0% (189/610) vs. non-equivalent 16.7% (102/610), i.e. the non-equivalent set was if anything *more* stable. The mid-accuracy judge moves when the criterion changes (so equivalent-rubric flips are a conservative floor on real instability), whereas the near-random judge responds to surface features regardless of rubric content, consistent with a judge below the competence floor.

D Head-to-Head System Comparison (Full Table)

Table 4 reports the full three-way *system* comparison summarized in Section 4.3, at the matched six-call budget.

E Feature Definitions

Table 5 defines all 13 features of the vector $\phi(x)$ in Eq. (5). Confidence values are the judge’s verbalized self-reported confidence in $[0, 1]$; entropies are computed over the empirical winner distribution across the relevant call set and normalized to $[0, 1]$.

Per-seed risk reporting. For every model–dataset– α setting we release, per seed: the accepted count, coverage, realized test error $\hat{R}_{\text{test}}$, the Clopper–Pearson upper bound on the calibration split, and whether $\hat{R}_{\text{test}} > \alpha$. To characterize variance rather than only central tendency, we resample the 40/30/30 split over many random seeds, up to several hundred paired splits per setting where feasible, and report coverage and selective risk as mean $\pm$ std across these splits alongside the pooled point estimates used in the main tables (pooling errors and accepts, rather than averaging per-seed ratios, so that zero-coverage seeds neither dominate nor are silently dropped). Aggregate seed robustness is high (coverage coefficient of variation $\approx 2.4\%$ at $\alpha=0.15$). We also report the empirical risk-violation frequency $\text{Pr}_{\text{seed}}(\hat{R}_{\text{test}} > \alpha)$ as our direct check on risk control; because the acceptance threshold is chosen data-dependently, this measured frequency is the quantity we validate against, and occasional violations at very low coverage (where only a handful of items are accepted) are expected and reported transparently.

F Exploratory: Cost-Aware Cascade

This section uses the **same strict 40/30/30 split as the main experiments** and calibrates each tier *conditionally* on the calibration items that actually reach it, with error budget $\delta_t = \delta/T$ (Section 3.3). We still label it exploratory because it uses a single labeled pool per dataset and we have not stress-tested the conditional calibration under distribution shift.

We evaluate a three-tier cascade (Qwen-1.5B $\rightarrow$ DeepSeek-V4 $\rightarrow$ GPT-5.5), averaging over 10 seeds (Table 6). Only RewardBench yields net savings: the cheap tiers certify almost every item, giving 89% paid-API savings at $\alpha=0.15$ (0.997 coverage, 0.908 accuracy) rising to 100% at $\alpha=0.30$. On the other three benchmarks the cheap tiers certify little that survives the strict risk bound, so items either escalate to GPT-5.5 or are certified only after paying the earlier tiers, and savings are negative at $\alpha=0.15$ (–11 to –14%); they turn positive only at the loose $\alpha=0.30$ (JudgeBench +8%, TL;DR +13%, LMarena +54%),

Table 4: Three-way system comparison: held-out AUROC at the matched six-call budget (20 seeds, strict 40/30/30 split), all five judges. CARE is our fused calibrator; SCOPE is bidirectional-consistency confidence; ToE is simulated-annotator agreement at $N=0$ dedicated annotators. Best per row in **bold**. CARE is best on 14 of the 17 judge–benchmark pairs; the three exceptions are all on JudgeBench for a Qwen judge, where SCOPE’s bidirectional score is marginally higher and the judge itself is near its accuracy floor.

Judge	Data	CARE	SCOPE	ToE
Qwen2.5-1.5B	JB	0.523	0.534	0.526
Qwen2.5-1.5B	TL;DR	0.583	0.570	0.525
Qwen2.5-1.5B	RB	0.713	0.425	0.492
Qwen2.5-1.5B	Arena	0.578	0.563	0.527
Qwen2.5-7B	JB	0.548	0.578	0.508
Qwen2.5-7B	TL;DR	0.636	0.616	0.504
Qwen2.5-7B	RB	0.687	0.605	0.507
Qwen2.5-7B	Arena	0.599	0.588	0.508
DeepSeek-V4	JB	0.603	0.561	0.510
DeepSeek-V4	TL;DR	0.675	0.616	0.519
DeepSeek-V4	RB	0.831	0.606	0.513
DeepSeek-V4	Arena	0.678	0.615	0.517
GPT-5.5	JB	0.834	0.695	0.526
GPT-5.5	TL;DR	0.667	0.629	0.511
GPT-5.5	RB	0.888	0.757	0.499
GPT-5.5	Arena	0.645	0.571	0.510
Qwen2.5-14B	JB	0.539	0.569	0.505

where the cheap tiers begin to accept at higher coverage. Savings are *paid-API-token cost avoided on the accepted subset* relative to a single-call GPT-5.5 judge, and exclude local GPU time, throughput, and latency.

G Exploratory: Cross-Dataset Transfer

We use the strict three-way split: the calibrator is fit on the *source* dataset (train+cal) and evaluated on a held-out *target* test split, averaged over 10 seeds (Table 7, GPT-5.5, $\alpha=0.15$). Two patterns emerge. Calibrators trained on *objective* benchmarks (JudgeBench, RewardBench) transfer at near-full coverage (0.98–1.00) but with target-dependent accuracy, from 0.960 (JudgeBench→RewardBench, both objective) down to 0.707 (JudgeBench→LMArena). Calibrators trained on *subjective* benchmarks transfer far more conservatively: an LMArena-trained calibrator accepts only 12.0% of JudgeBench, 2.9% of TL;DR, and 32.0% of RewardBench, at accuracies of 0.60–0.80. The open-ended-preference signal is thus the least portable, since the LMArena source yields the lowest coverage of any row, and transfer is most accurate when both source and target carry objective labels.

H Risk–Coverage Curves

I Signal-Level Accuracy Gaps

The per-benchmark accuracy gaps summarized by Figure 2 (in the main body) are tabulated below.

J Ensemble Baselines and Feature-Group Ablation

K Length Bias by Judge and Benchmark

Table 11 reports how often each judge selects the *longer* of the two responses, recovered by aligning the feature traces with the raw responses in `data/` (the `length_gap` feature stores only the unsigned magnitude, so direction is recomputed here via `scripts/compute_signal_gaps.py`). Two patterns emerge. First, length preference is benchmark-driven: on the objectively-labeled JudgeBench and RewardBench every judge is near 50% (no systematic length preference), whereas on the preference benchmarks TL;DR and LMArena all judges lean long, tracking the human label’s own length correlation. Second, on LMArena the lean toward longer responses *decreases* with judge accuracy (67% for Qwen2.5-1.5B down to 53% for GPT-5.5), consistent with competence-gating: a stronger judge relies less on the length heuristic. This also explains why the length feature group is near-chance in the ablation (Table 10).

Table 5: The 13 features of $\phi(x)$ (Eq. 5), grouped by family.

Feature	Definition
<i>Protocol: position (swap)</i>	
swap_consistency	$\mathbb{I}[\text{verdict unchanged after response reorder}]$ (Eq. 3).
swap_conf_gap	$ \text{conf}_{\text{orig}} - \text{conf}_{\text{swap}} $: confidence change under reorder.
<i>Protocol: rubric</i>	
rubric_vote_share	majority winner share over $K=3$ rubric paraphrases (Eq. 1).
rubric_entropy	normalized entropy of the winner distribution over the K rubrics.
rubric_flip	$\mathbb{I}[\text{any rubric paraphrase changes the winner}]$ (Eq. 2).
<i>Protocol: self-consistency</i>	
self_vote_share	majority winner share over $S=3$ samples (Eq. 4).
self_entropy	normalized entropy of the winner distribution over the S samples.
<i>Confidence</i>	
confidence	aggregate verbalized confidence pooled across the perturbation calls.
base_conf	verbalized confidence of the deterministic base verdict ($\tau=0$).
mean_conf	mean verbalized confidence across all six calls.
std_conf	standard deviation of verbalized confidence across the six calls.
<i>Length</i>	
length_gap	normalized token-length difference $ a_1 - a_2 $ between the two responses.
score_margin	difference between the base verdict's confidence and 0.5 (decisiveness).

Table 6: Cost-aware cascade (strict 40/30/30 split, conditional per-tier calibration, $\delta_t=\delta/T$, 10 seeds). Cov/Acc on held-out test; Save is paid-API-token cost avoided vs. single-call GPT-5.5.

Dataset	α	Cov.	Acc.	Save
JudgeBench	0.15	0.998	0.913	-14%
	0.30	0.998	0.881	8%
TL;DR	0.15	0.087	0.752	-14%
	0.30	0.866	0.758	13%
RewardBench	0.15	0.997	0.908	89%
	0.30	0.999	0.768	100%
LMArena	0.15	0.069	0.617	-11%
	0.30	0.821	0.748	54%

Table 7: Cross-dataset transfer for GPT-5.5 ($\alpha=0.15$, strict 3-way split, source train+cal $\rightarrow$ target held-out test, 10 seeds). Subjective sources (LMArena, TL;DR) yield precise but low-coverage transfer; objective sources (JudgeBench, RewardBench) yield higher coverage at lower accuracy.

Source $\rightarrow$ Target	Cov.	Acc.
TL;DR $\rightarrow$ JudgeBench	0.294	0.889
TL;DR $\rightarrow$ RewardBench	0.628	0.881
TL;DR $\rightarrow$ LMArena	0.277	0.730
JudgeBench $\rightarrow$ TL;DR	0.991	0.732
JudgeBench $\rightarrow$ RewardBench	0.999	0.960
JudgeBench $\rightarrow$ LMArena	0.991	0.707
RewardBench $\rightarrow$ JudgeBench	0.996	0.913
RewardBench $\rightarrow$ TL;DR	0.986	0.733
RewardBench $\rightarrow$ LMArena	0.985	0.709
LMArena $\rightarrow$ JudgeBench	0.120	0.596
LMArena $\rightarrow$ TL;DR	0.029	0.576
LMArena $\rightarrow$ RewardBench	0.320	0.798

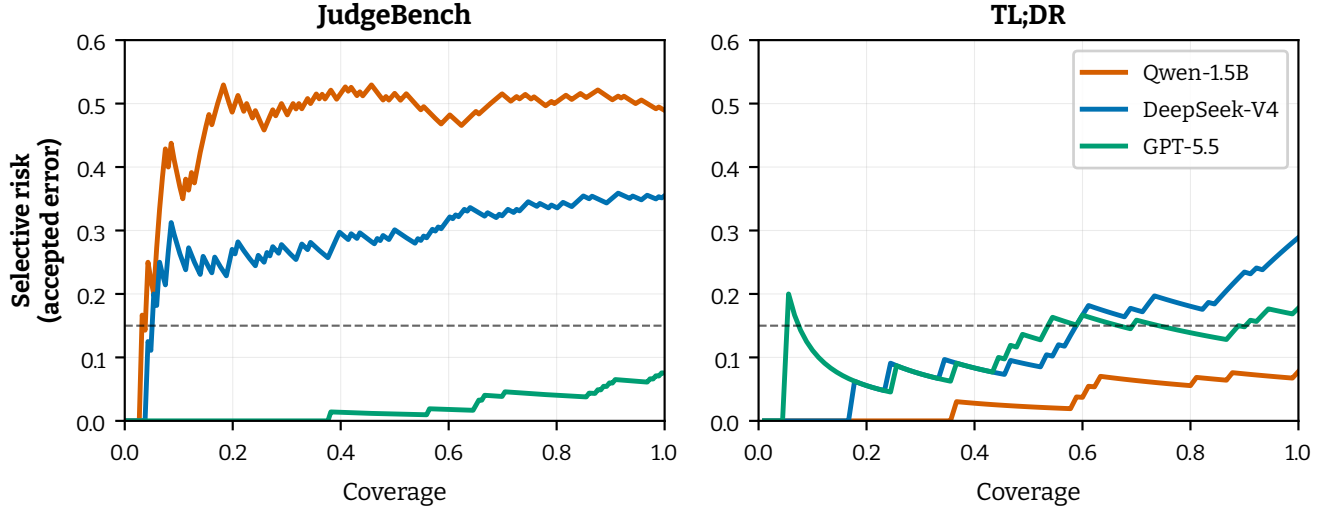


Figure 3: Held-out risk-coverage curves (logistic fusion); lower selective risk at a given coverage is better, and the dashed line marks the $\alpha=0.15$ risk budget. Curves are shown per judge on JudgeBench and TL;DR.

Table 8: Signal-level accuracy on stable vs. unstable subsets, recomputed from the per-item feature traces (`scripts/compute_signal_gaps.py`) for all four primary judges. Gap = stable – unstable accuracy. Gaps are positive throughout for the mid-accuracy and frontier judges; the only non-positive entries are Qwen2.5-1.5B on RewardBench (rubric -0.109 , position -0.110), the below-competence case where both subsets sit near chance and stability is uninformative (Section 4.4).

Signal	Model	Dataset	Stable	Unstable	Gap
Rubric	Qwen2.5-1.5B	JudgeBench	0.511	0.459	+0.052
	Qwen2.5-1.5B	RewardBench	0.724	0.833	−0.109
	DeepSeek-V4	JudgeBench	0.689	0.648	+0.041
	DeepSeek-V4	TL;DR	0.708	0.555	+0.153
	DeepSeek-V4	RewardBench	0.937	0.664	+0.274
	DeepSeek-V4	LMarena	0.749	0.478	+0.271
	Qwen2.5-7B	LMarena	0.667	0.496	+0.171
	GPT-5.5	RewardBench	0.962	0.692	+0.270
Position	Qwen2.5-1.5B	JudgeBench	0.553	0.469	+0.084
	Qwen2.5-1.5B	RewardBench	0.713	0.823	−0.110
	DeepSeek-V4	JudgeBench	0.728	0.575	+0.153
	DeepSeek-V4	TL;DR	0.746	0.518	+0.228
	DeepSeek-V4	LMarena	0.772	0.528	+0.244
	Qwen2.5-7B	TL;DR	0.739	0.523	+0.216
	GPT-5.5	JudgeBench	0.929	0.621	+0.308
	GPT-5.5	LMarena	0.729	0.524	+0.205

Table 9: Ensemble baselines vs. CARE (held-out AUROC, 20 seeds), all four non-frontier/frontier judges. CARE is best or tied-best in 12 of 16 pairs; weighted vote is competitive for GPT-5.5 (it uses confidence implicitly) but has no risk-control mechanism.

Model	Data	CARE	SCOPE	MajVote	WtdVote
Qwen2.5-1.5B	JB	0.523	0.534	0.505	0.509
	TL;DR	0.583	0.570	0.544	0.494
	RB	0.713	0.425	0.538	0.620
	Arena	0.578	0.563	0.559	0.494
Qwen2.5-7B	JB	0.548	0.578	0.555	0.531
	TL;DR	0.636	0.616	0.523	0.562
	RB	0.687	0.605	0.602	0.624
	Arena	0.599	0.588	0.569	0.531
DeepSeek-V4	JB	0.603	0.561	0.585	0.581
	TL;DR	0.675	0.616	0.609	0.618
	RB	0.831	0.606	0.675	0.810
	Arena	0.678	0.615	0.612	0.616
GPT-5.5	JB	0.834	0.695	0.583	0.806
	TL;DR	0.667	0.629	0.570	0.673
	RB	0.888	0.757	0.600	0.899
	Arena	0.645	0.571	0.568	0.643

Table 10: Feature-group ablation (held-out AUROC, 20 seeds). “FULL” = all 13 features; each ablation uses only that group. Confidence dominates for GPT-5.5; protocol features carry the Qwen judges; length is near-chance everywhere.

Model	Data	FULL	protocol	conf	length
Qwen2.5-1.5B	JB	0.523	0.534	0.495	0.497
	TL;DR	0.583	0.580	0.581	0.536
	RB	0.713	0.659	0.631	0.609
	Arena	0.578	0.579	0.587	0.535
Qwen2.5-7B	JB	0.548	0.557	0.537	0.506
	TL;DR	0.636	0.629	0.634	0.535
	RB	0.687	0.615	0.685	0.493
	Arena	0.599	0.603	0.587	0.520
DeepSeek-V4	JB	0.603	0.586	0.623	0.509
	TL;DR	0.675	0.660	0.683	0.548
	RB	0.831	0.691	0.828	0.622
	Arena	0.678	0.654	0.680	0.531
GPT-5.5	JB	0.834	0.692	0.847	0.510
	TL;DR	0.667	0.636	0.673	0.519
	RB	0.888	0.782	0.906	0.669
	Arena	0.645	0.598	0.647	0.505

Table 11: Percentage of decisions selecting the *longer* response (ties excluded), by judge and benchmark. Values near 50% indicate no systematic length preference; “–” marks judge–benchmark pairs not run.

Judge	JB	TL;DR	RB	Arena
Qwen2.5-1.5B	52	60	57	67
DeepSeek-V4	46	59	48	66
Qwen2.5-7B	48	58	46	61
GPT-5.5	44	58	43	53
Qwen2.5-14B	47	–	–	–